# Cutting the Cord: Trust Without the TA

Security Analysis and Improvements of a TUAV-Assisted Key Management Scheme for Infrastructure-Less IoVs

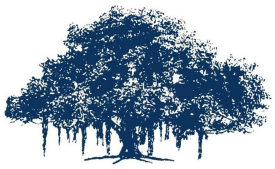
CS8.501 – Research in Information Security
Project Report


Submitted by:

| | |
|---|---|
| Swaraj Kumar | Roll Number: 2025202016 |
| Hritik Ranjan | Roll Number: 2025202031 |
| Harsh Raj | Roll Number: 2025202005 |

Department: Computer Science and Information Security


Date of Submission: November 17, 2026



INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

HYDERABAD


International Institute of Information Technology, Hyderabad

# Abstract

Our project is based on the paper "Secure and Efficient Authenticated Key Management Scheme for UAV-Assisted Infrastructure-Less IoVs" by Tan, Zheng and Vijayakumar (IEEE T-ITS, 2023) [1]. The paper uses a tethered UAV (TUAV), powered by an emergency vehicle, as a base station in a disaster area. A trusted authority (TA), reached by satellite, authenticates a group of UAVs, and the TUAV gives them a group key built with the Chinese Remainder Theorem (CRT).

We studied the scheme and then checked its security claims against its own equations. We found and proved three weaknesses: (F1) the TA can sign as any UAV; (F2) the CRT group key leaks every member's key, even to an outsider who only listens to two broadcasts; (F3) one leaked long-term secret opens all past group keys. We then designed four contributions: (C1) authentication with the TA offline and a separately sealed key for every member; (C2) handover between TUAVs; (C3) voting out a compromised TUAV; (C4) splitting the TUAV's key across its tether. We also built a simulator of the base scheme and an implementation of our design; both are available online.

# Contents

**Abstract** **1**

**1 Introduction** **4**
- 1.1 Problem statement . . . 4
- 1.2 Summary of our work . . . 4
- 1.3 Team and individual contributions . . . 5

**2 Background** **5**

**3 The base paper** **6**
- 3.1 System model . . . 6
- 3.2 The protocol . . . 6
- 3.3 Related work discussed in the paper . . . 8
- 3.4 What the paper claims . . . 8

**4 Weaknesses of the base scheme** **9**
- 4.1 F1: the TA can sign as any UAV . . . 9
- 4.2 F2: the CRT group key leaks . . . 9
- 4.3 F3: no forward secrecy . . . 11

**5 Our contributions** **11**
- 5.1 C1: authentication with the TA offline . . . 11
- 5.2 C2: handover between TUAVs . . . 13
- 5.3 C3: voting out a compromised TUAV . . . 14
- 5.4 C4: the tether-split key . . . 15
- 5.5 A1–A7: fixing our own first design . . . 17

**6 Results** **18**
- 6.1 Security comparison . . . 18

6.2 Cost from the paper's own constants . . . . . 18
6.3 Simulator and implementation . . . . . 18

**7 Related work and conclusion** **20**
7.1 Related work . . . . . 20
7.2 Limitations . . . . . 21
7.3 Conclusion . . . . . 21

**References** **21**

# List of Figures

1.1 How our results fit together. Left: the three weaknesses we prove in the base scheme; all are removed by contribution C1. After C1 the TA is offline, so the TUAV becomes the trusted party. This leaves three gaps (third column), which are closed by C2, C3 and C4. . . . . 4
3.1 System model of the TUAV-assisted infrastructure-less IoV, reproduced from Fig. 1 of the base paper [1]. The UAV network (top left) talks to the tethered UAV, which is powered by the ECRV. The ECRV reaches the TA through the satellite network. Vehicles (bottom left) communicate through the UAVs because the RSU is out of service. . . . . 6
3.2 The five phases of the base scheme, in order. The labels on the left (IS, US, GA, GD, DU) are the step names used in Section IV of the paper. Solid arrows show the normal order; the dashed arrow shows that a key update repeats the key-distribution step. . . . . 7
3.3 Message flow of the base scheme between a UAV, the TUAV and the TA. Time runs from top to bottom. Solid arrows are local radio messages; dashed arrows cross the satellite link. Boxes show what each entity computes. . . . . 8
3.4 Group-authentication time against the number of UAVs, plotted from the cost formulas in Section VI-A of the base paper for the four schemes it compares with and for itself. The base paper's line counts only the TUAV side; the TA's three pairings per UAV are not included. . . . . 9
4.1 Flowchart of the outsider attack. The attacker records the coefficient sets before and after one membership change, gets the changed member's key from the two sums, removes the mask, and finds all keys and both group keys. It needs no registration and no secret. . . . . 10
5.1 Message flow of C1 between the TUAV and $n$ UAVs. Time runs from top to bottom; boxes show local computation. The TA (centre) is never contacted. Compared with Figure 3.3, there is no satellite link, no pairing and no CRT polynomial. . . . . 12
5.2 Message flow of the C2 handover between the old TUAV $\mathcal{T}_a$, the moving member $\mathcal{M}$ and the new TUAV $\mathcal{T}_b$. Time runs from top to bottom. The two signatures in bold (by the member in HO2 and by the new TUAV in HO3) are needed so that a captured old TUAV cannot misuse a ticket. . . . . 14
5.3 Quorum eviction (C3). The neighbours $T_1, T_2, T_3$ get evidence against the compromised TUAV $T_7$ (dashed arrows) and each signs a vote. At least $t$ different votes form an eviction certificate. It is delivered to the members of $T_7$, who check it and join a neighbour. . . . . 15

5.4 Flowchart of how a UAV checks an eviction certificate against the four conditions V1–V4. Only if all conditions hold does the UAV leave its TUAV; otherwise it ignores the certificate. . . . . 15
5.5 The tether-split key (C4). The drone holds the share $a$ and a secure chip in the ECRV holds the share $g$; they are connected by the tether. The public key $pk = aP + gP$ is unchanged, and no single device holds the full key $a + g$. . . . . 16
5.6 Joint signing over the tether (steps J1–J5). Messages 1–4 travel on the tether between the drone and the chip. The final signature $(R, v)$ is sent by radio and is checked in the usual way. The commitment in message 1 forces the drone to fix its value $R_A$ before it sees the chip's. . . . . 16
5.7 What happens after the tether is cut. The chip stops signing at once, the ECRV broadcasts the stored "out of service" message, old beacons expire after a short period $\delta$, and the UAVs join a neighbouring TUAV with fresh keys. . . . . 17
6.1 Traffic needed to distribute one group key against the number of UAVs $n$ (logarithmic vertical axis). Red: the base scheme, which sends all polynomial coefficients to each UAV. Blue: C1, one broadcast with about 80 bytes per member. The marked points are the values for $n = 120$. . . . . 18
6.2 Screenshot of our simulator of the base scheme. Top: status of the system (RSU destroyed, TUAV online, 8 UAVs registered and authenticated, group key of epoch 1). Left: the network as in the paper's system model. Right: the registered UAVs with hidden permanent identities and temporary identities. . . . . 19
6.3 Screenshot of the simulator after group key distribution (steps GD1–GD6). All 8 members recover the same group key from one broadcast of 303 bytes. The note at the bottom records that we read $\sigma_i$ as an integer, because the paper uses it as a modulus. . . . . 19
6.4 Measured time to authenticate $n$ UAVs and give them the group key, on our implementation of both schemes (same laptop, pure Python, BN128 curve). The vertical axis is logarithmic. The base scheme needs three pairings per UAV at the TA; our design needs none, and is about 60 times faster at every size. . . . . 20

# 1 Introduction

## 1.1 Problem statement

After an earthquake or a flood, roadside units (RSUs) and mobile towers often stop working. Rescue vehicles and drones still need to communicate, and this communication must be authenticated and secret. The base paper [1] solves this with a **tethered UAV (TUAV)**: a drone that gets power through a cable from an emergency vehicle (ECRV) and works as a flying base station. Battery UAVs extend its coverage. A trusted authority (TA), reached through a satellite link, checks the UAVs, and the TUAV gives them a common group key.

Our work has two goals: (i) check whether the scheme really gives the security it claims, and (ii) design improvements where it does not.

## 1.2 Summary of our work

| | |
|---|---|
| **F1** | The TA can produce a valid login request for any UAV (key escrow). |
| **F2** | The CRT group key leaks: a removed UAV reads new keys, a new UAV reads old keys, and an outsider who records two broadcasts gets every key. |
| **F3** | No forward secrecy: one leaked long-term secret opens all past group keys. |
| **C1** | Authentication with the TA offline, and one sealed copy of the group key per member. Fixes F1, F2 and F3. |
| **C2** | Handover of a UAV between two TUAVs, without the TA. |
| **C3** | Quorum eviction: $t$ TUAVs vote out a compromised TUAV. |
| **C4** | Tether-split key: a captured TUAV holds only half a key and cannot sign. |
| **A1–A7** | Seven flaws we found in our own first design, and their fixes. |

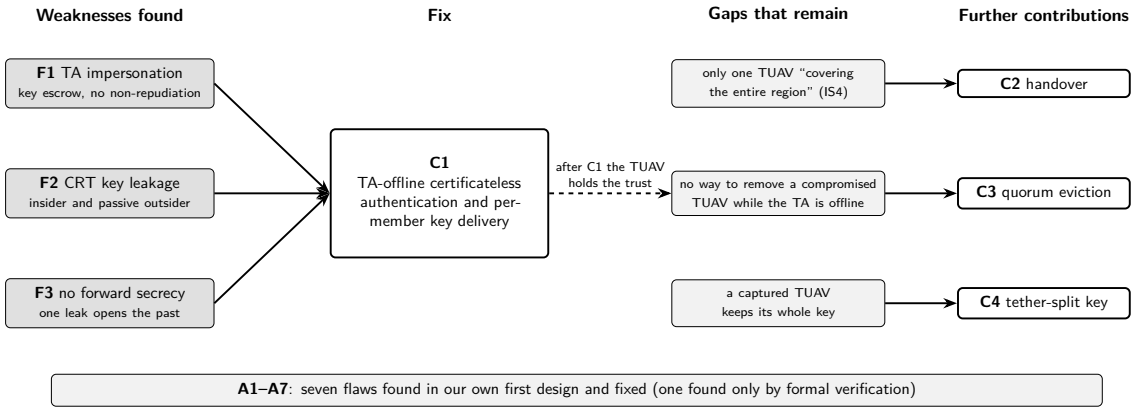



**Figure 1.1:** How our results fit together. Left: the three weaknesses we prove in the base scheme; all are removed by contribution C1. After C1 the TA is offline, so the TUAV becomes the trusted party. This leaves three gaps (third column), which are closed by C2, C3 and C4.

### 1.3 Team and individual contributions

| Member | Roll number | Main responsibilities |
|---|---|---|
| Swaraj Kumar | 2025202016 | study of the base paper; overall framework of our design; simulator; compiling the report |
| Hritik Ranjan | 2025202031 | mathematical analysis of F1, F2 and F3; design of C1 and its security analysis |
| Harsh Raj | 2025202005 | design and security analysis of C2, C3 and C4; the study A1–A7; results |

# 2 Background

This chapter lists the tools used in the rest of the report.

| Symbol | Meaning |
|---|---|
| $\langle id_i, s_i \rangle$ | base paper: real identity and long-term secret of UAV $i$, issued and stored by the TA |
| $r_i,\ \xi_i = h_1(r_i)$ | base paper: random value of the UAV and its hash; $\xi_i$ is sent in the clear |
| $\Gamma_i$ | base paper: credential (login request) of UAV $i$ |
| $\sigma_i = h_1(s_i \xi_i) P$ | base paper: "decrypting key" of UAV $i$ |
| $\Lambda(x)$, $\{a_k\}$ | base paper: broadcast polynomial and its public coefficients |
| $\kappa_{tu}$ | base paper: group key |
| $s,\ P_{\text{pub}} = sP$ | our design: master key and public key of the TA |
| $x,\ d,\ sk = x + d,\ pk$ | our design: user's own secret, partial key from the TA, signing key, public key |
| $e,\ E = eP$ | our design: fresh Diffie–Hellman values, deleted after use |
| $GK,\ \nu$ | our design: group key and its version number |
| KDF, Enc | key-derivation function and authenticated encryption |
| $a,\ g$ | C4: the drone's share and the chip's share of the TUAV key |

**Hard problems.** Let $\mathbb{G}$ be a group of prime order $q$ with generator $P$. *ECDLP*: given $P$ and $aP$, find $a$. *CDH*: given $P$, $aP$ and $bP$, find $abP$. Both are assumed to be hard. A CDH problem is hard only if $a$ and $b$ are unknown.

**Chinese Remainder Theorem (CRT).** Let $m_1, \ldots, m_n$ be pairwise coprime and $M = \prod_i m_i$. For any values $c_1, \ldots, c_n$ there is exactly one $X$ in $[0, M)$ with $X \equiv c_i \pmod{m_i}$ for all $i$. It is $X = \sum_i c_i \omega_i \varphi_i \bmod M$ with $\omega_i = M/m_i$ and $\varphi_i \equiv \omega_i^{-1} \pmod{m_i}$. The base paper uses the case where all $c_i$ are the group key.

**Vieta's formulas.** If $p(x) = \prod_{i=1}^{n}(x - \sigma_i) = x^n + c_{n-1}x^{n-1} + \cdots + c_0$, then $c_{n-1} = -(\sigma_1 + \cdots + \sigma_n)$ and $c_0 = (-1)^n \sigma_1 \cdots \sigma_n$. So the coefficients of a product polynomial are simple functions of its roots.

**Schnorr signature [4].** Key pair $(sk, pk = sk \cdot P)$. To sign $m$: pick random $r$, set $R = rP$, $c = H(m \,\|\, R)$, $v = r + c \cdot sk$. The signature $(R, v)$ is valid if $vP = R + c \cdot pk$. It is unforgeable if the ECDLP is hard (shown with the forking lemma [5]).

**Security properties we check.**

- *Key-escrow resilience*: the authority that issues keys cannot sign in a user's name. Girault [7] grades authorities: level 1 knows the users' keys; level 2 does not, but can issue a false second credential; level 3 cannot do even that without proof.
- *Non-repudiation*: nobody except the signer could have produced a signature.
- *Revocation*: a removed member learns nothing about later group keys. *Backward secrecy*: a new member learns nothing about earlier group keys.
- *Forward secrecy*: old keys stay secret even if long-term secrets leak later.

# 3 The base paper

## 3.1 System model

The system has four parts (Figure 3.1): the **TA**, a trusted server that sets up the system and verifies users; the **TUAV**, a drone powered through a cable by the **ECRV**; the **UAV network** of battery drones; and the **vehicles**. The RSUs are assumed to be out of service, and the TA is reached through satellites.

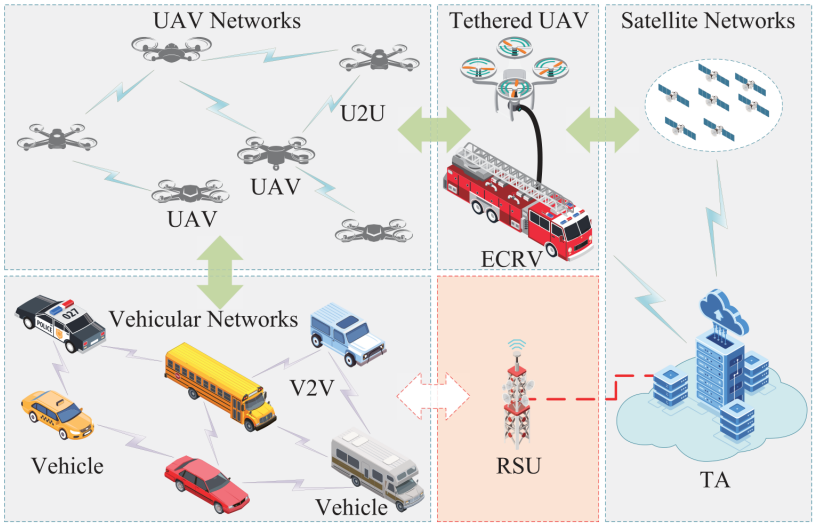



**Figure 3.1:** System model of the TUAV-assisted infrastructure-less IoV, reproduced from Fig. 1 of the base paper [1]. The UAV network (top left) talks to the tethered UAV, which is powered by the ECRV. The ECRV reaches the TA through the satellite network. Vehicles (bottom left) communicate through the UAVs because the RSU is out of service.

## 3.2 The protocol

The scheme uses a pairing $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_1 \to \mathbb{G}_2$ on groups of prime order $q$ with generator $P$, and hash functions $\mathsf{H}_1, h_1, \ldots, h_4$. It has five phases (Section IV of the paper). We give the equations that matter for our analysis.

Figure 3.2 gives an overview.

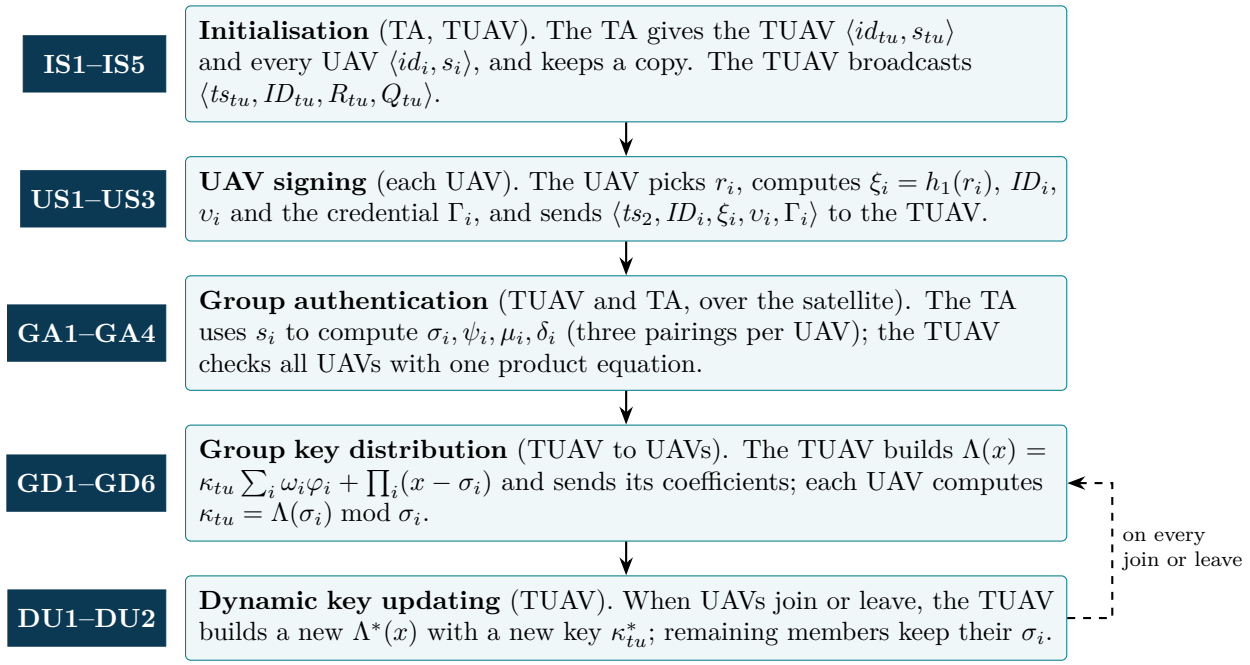



**Figure 3.2:** The five phases of the base scheme, in order. The labels on the left (IS, US, GA, GD, DU) are the step names used in Section IV of the paper. Solid arrows show the normal order; the dashed arrow shows that a key update repeats the key-distribution step.

**Initialisation (IS1–IS5).** The TA gives the TUAV $\langle id_{tu}, s_{tu}\rangle$ and every UAV $\langle id_i, s_i\rangle$, and *stores all of them.* The TUAV broadcasts $ID_{tu}$, $R_{tu} = r_{tu} h_2(ts_{tu}, s_{tu})$ and $Q_{tu} = s_{tu} h_2(ID_{tu}, r_{tu}) P$.

**UAV signing (US1–US3).** UAV $i$ picks a random $r_i$ and computes $\xi_i = h_1(r_i)$, $ID_i = h_3(id_i, ts_2, \xi_i)$, $\upsilon_i = h_4(ID_i, ID_{tu}, ts_2, \xi_i)$ and the credential

$$\Gamma_i = \big[s_i\, h_3(ID_i, ts_2, s_i)\, R_{tu} - h_1(s_i \xi_i)\big] P - \upsilon_i\, h_1(r_i)\, Q_{tu}. \tag{3.1}$$

It sends $\langle ts_2, ID_i, \xi_i, \upsilon_i, \Gamma_i\rangle$ to the TUAV in the clear.

**Group authentication (GA1–GA4).** The TUAV sends the requests to the TA over the satellite. For every UAV the TA uses the stored $s_i$ to compute $\sigma_i = h_1(s_i \xi_i) P$ and three pairings $\psi_i, \mu_i, \delta_i$. The TUAV then checks all $n$ UAVs with one equation, $\prod_i \psi_i \, (\prod_i \delta_i)^{h_2(ID_{tu}, r_{tu})} = \prod_i \mu_i^{R_{tu}}$.

**Group key distribution (GD1–GD6).** With $\omega_i = \prod_{j \neq i} \sigma_j$ and $\varphi_i \equiv \omega_i^{-1} \bmod \sigma_i$, the TUAV picks a group key $\kappa_{tu}$ and builds

$$\Lambda(x) = \kappa_{tu} \sum_{i=1}^{n} \omega_i \varphi_i + \prod_{i=1}^{n} (x - \sigma_i) = \sum_{k=0}^{n} a_k x^k. \tag{3.2}$$

It sends the coefficients $\{a_k\}$ to each UAV in the clear. UAV $i$ computes $\kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i$. This works by the Chinese Remainder Theorem: at $x = \sigma_i$ the product is zero, and the remaining number leaves remainder $\kappa_{tu}$ modulo every $\sigma_i$.

**Dynamic key updating (DU1–DU2).** When UAVs join or leave, the TUAV builds a new polynomial $\Lambda^*(x)$ with a new key. Members who stay keep their old $\sigma_i$.

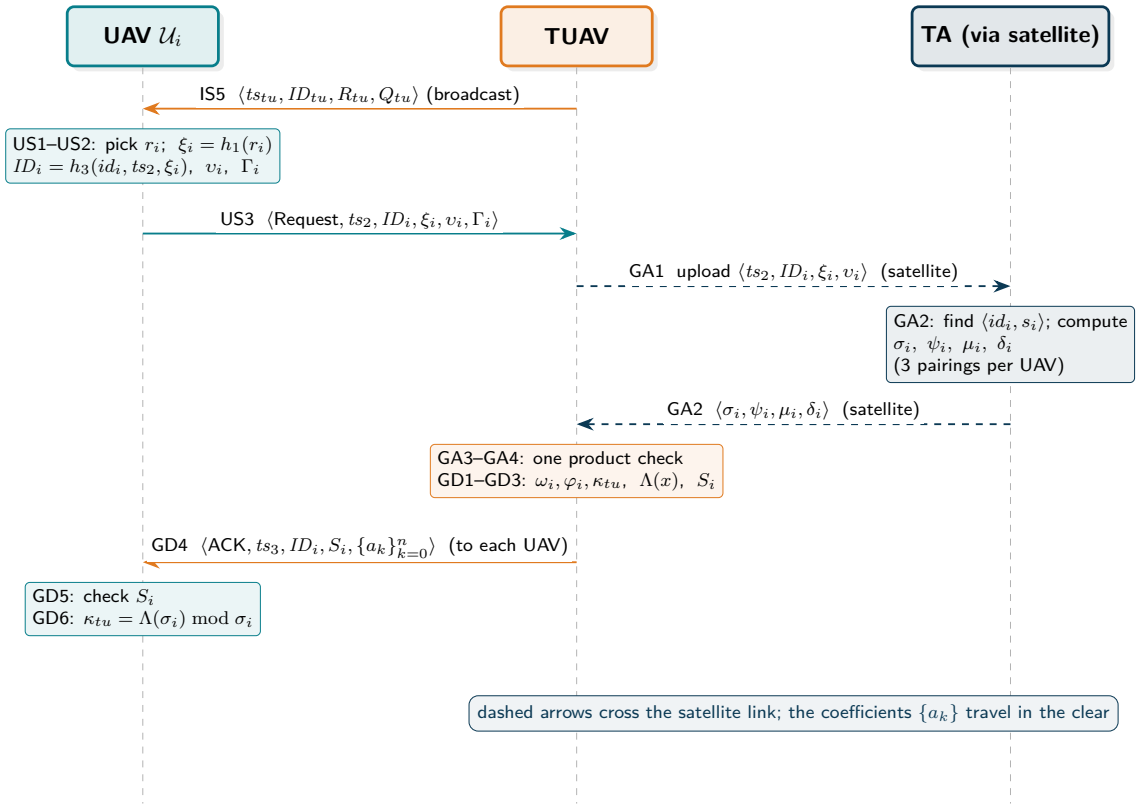



**Figure 3.3:** Message flow of the base scheme between a UAV, the TUAV and the TA. Time runs from top to bottom. Solid arrows are local radio messages; dashed arrows cross the satellite link. Boxes show what each entity computes.

## 3.3 Related work discussed in the paper

The paper reviews earlier authentication schemes for vehicle networks: schemes for authenticated key agreement, identity-based and attribute-based schemes, lattice-based schemes, batch authentication schemes, and UAV-assisted schemes. Its main point is that all of them need a working roadside infrastructure, which is missing in a disaster area. It compares itself with four of them (Mei et al., Kumar et al., Ali et al., Xu et al.) on security properties and on cost.

## 3.4 What the paper claims

The paper proves five theorems (unforgeability, anonymity, replay resistance, conditional privacy, session key establishment). Its Table II ticks eleven properties for the scheme, including **key escrow resilience**, **forward secrecy** and **non-repudiation**. For the group key it says (Section IV-D): the scheme "supports multi-device revocation and new device participation", and

> "... the malicious entities that successfully collect the coefficient set $\{a_i\}_{i\in[0,n]}$ can not acquire the $\kappa_{tu}$ without the correct decrypting key $\sigma_i$."

For cost, the paper reports $1.2524n + 0.9054$ ms for group authentication (151 ms for 120 UAVs). This counts only the TUAV side; the TA's pairings are done "prior to the verification of TUAV".

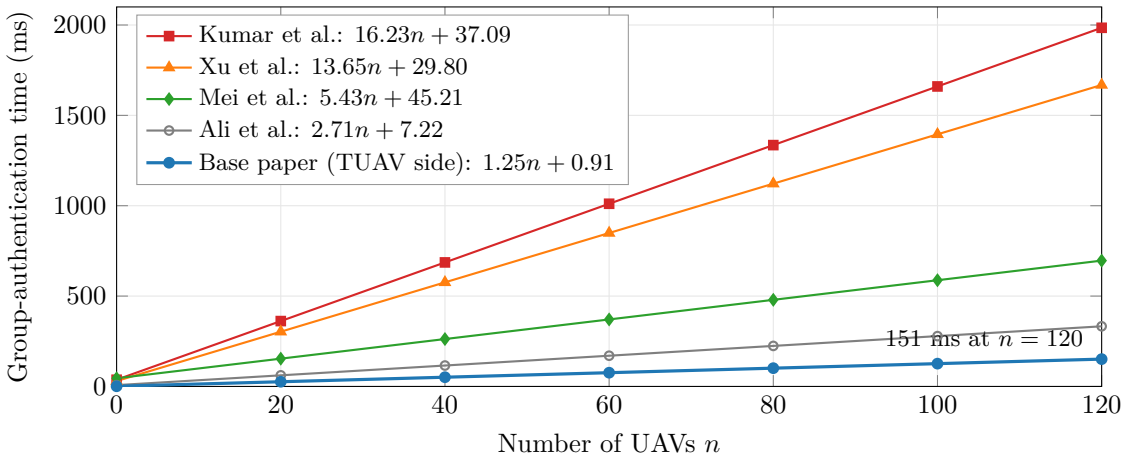



**Figure 3.4:** Group-authentication time against the number of UAVs, plotted from the cost formulas in Section VI-A of the base paper for the four schemes it compares with and for itself. The base paper's line counts only the TUAV side; the TA's three pairings per UAV are not included.

*Remark* 3.1. The paper writes $\sigma_i = h_1(s_i\xi_i)P$, a curve point, but uses it as a modulus. So any implementation must read $\sigma_i$ as an integer. All our results hold for any such reading.

# 4 Weaknesses of the base scheme

## 4.1 F1: the TA can sign as any UAV

In Eq. (3.1) the only secret is $s_i$, and the TA stores it (IS2). The random $r_i$ is used only as $h_1(r_i) = \xi_i$, and $\xi_i$ is sent in the clear. Everything else $(ID_i, v_i, ts_2, R_{tu}, Q_{tu})$ is public.

**Theorem 4.1** (TA forgery)**.** *Anyone who knows $\langle id_i, s_i\rangle$ can create a request that the TUAV and TA accept as UAV $i$, and can then get the group key.*

*Proof.* Pick any $\xi'$ and a fresh $ts'$. Compute $ID' = h_3(id_i, ts', \xi')$, $v' = h_4(ID', ID_{tu}, ts', \xi')$ and $\Gamma' = [s_i h_3(ID', ts', s_i)R_{tu} - h_1(s_i\xi')]P - v'\xi' Q_{tu}$. This is exactly an honest request with $\xi'$ in place of $h_1(r_i)$, so the check of GA3 holds for the same reason as for an honest UAV. The forger then computes $\sigma' = h_1(s_i\xi')P$ and $\kappa_{tu} = \Lambda(\sigma') \bmod \sigma'$. □

**Effect.** The TA, or anyone who steals its database, can log in as any UAV. So *key escrow resilience* and *non-repudiation* (Table II) do not hold. The paper calls $r_i$ the UAV's "partial key" (proof of its Theorem 4), but $r_i$ adds no secret because only the public $\xi_i$ is used.

## 4.2 F2: the CRT group key leaks

In Eq. (3.2) only the constant term is hidden by $C = \kappa_{tu}\sum_i \omega_i\varphi_i$. All other coefficients are those of $\prod_i(x - \sigma_i)$.

**Theorem 4.2** (Insider key recovery)**.** *A member $j$ who knows only its own $\sigma_j$ and the public coefficients can compute every member's key $\sigma_i$.*

*Proof.* At $x = \sigma_j$ the product is zero, so $\Lambda(\sigma_j) = C$. The member subtracts this value and gets $\Lambda(x) - C = \prod_i(x - \sigma_i)$. This is a known polynomial with integer coefficients, and its roots are the

keys. Integer roots can be found quickly with standard methods (for example LLL factoring [10]); this is not a brute-force search. □

**Corollary 4.3** (Revocation and backward secrecy fail)**.** *(a) A UAV that is removed already knows the keys $\sigma_k$ of the members who stay, and these keys do not change (DU2). So it computes the new key $\kappa^*_{tu} = \Lambda^*(\sigma_k) \bmod \sigma_k$. (b) A UAV that joins learns the old members' keys in the same way and can open recorded old broadcasts, $\kappa_{tu} = \Lambda(\sigma_j) \bmod \sigma_j$.*

**Theorem 4.4** (Outsider key recovery)**.** *Let $\Lambda$ and $\Lambda'$ be the broadcasts before and after one UAV with key $\sigma$ leaves (or joins), with at least two members in common. An eavesdropper who knows only the two coefficient sets can compute $\sigma$, every other key, and both group keys.*

*Proof.* Let $p(x) = \prod_{\text{old}}(x - \sigma_i)$ and $q(x) = \prod_{\text{new}}(x - \sigma_i)$. The eavesdropper knows all coefficients of $p$ and $q$ except the constants $p_0, q_0$. By Vieta's formulas, the second coefficient of each polynomial is minus the sum of its keys. So the difference of the two sums is $\sigma$. For a leave, $p(x) = (x - \sigma)q(x)$; comparing the coefficients of $x$ gives $p_1 = q_0 - \sigma q_1$, so

$$q_0 = p_1 + \sigma\, q_1.$$

Now $q$ is fully known. Its roots are the remaining keys, and $p = (x - \sigma)q$ is known too. With any key, the attacker computes both group keys like a member. For a join the roles are swapped: $p_0 = q_1 + \sigma p_1$. □

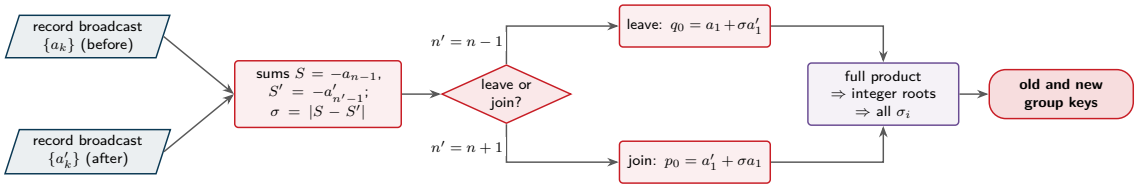



**Figure 4.1:** Flowchart of the outsider attack. The attacker records the coefficient sets before and after one membership change, gets the changed member's key from the two sums, removes the mask, and finds all keys and both group keys. It needs no registration and no secret.

**Proposition 4.5** (Several changes at once)**.** *Suppose $m$ UAVs leave in one update and at least $m + 1$ members stay (or $m$ UAVs join a group of at least $m + 1$). Then the two broadcasts again give all keys.*

*Proof.* For a leave write $p = q \cdot d$, where $q$ is the product over the $r$ members who stay and $d(x) = x^m + d_{m-1}x^{m-1} + \cdots + d_0$ is the product over the leavers. Compare the coefficients of $x^{n-j}$ for $j = 1, \ldots, m$ (with $n = r + m$):

$$d_{m-j} = p_{n-j} - \sum_{i=1}^{j} q_{r-i}\, d_{m-j+i}.$$

The right side uses only coefficients that are not masked ($n - j \geq 1$ and $r - i \geq r - m \geq 1$) and values of $d$ already found. So $d$ is computed from the top down. The coefficient of $x^m$ then gives the hidden constant $q_0 = p_m - \sum_{i \geq 1} q_i\, d_{m-i}$. Now $d$ and $q$ are known, and their roots are all the keys. A join is the same with the two broadcasts swapped. □

**Effect.** Theorem 4.4 contradicts the sentence quoted in Chapter 3: someone who only collects the coefficients *can* get $\kappa_{tu}$. Corollary 4.3 contradicts "supports multi-device revocation and new device participation".

**Scope.** The outsider attack is proved for one join or one leave. The same algebra also works when several UAVs leave together (or join together), as long as more members stay than change. Two cases are open: a single broadcast alone, and one update that mixes joins and leaves. We checked the attack with a program at the paper's size (160-bit keys, 12 UAVs). CRT-based group keys are known to be weak when a member is corrupted [9]; our outsider attack needs no corrupted member. Among the 94 papers that cite the base paper we found none that reports it.

**Root cause.** The scheme publishes all coefficients of $\prod_i (x - \sigma_i)$ except the last one. These coefficients are functions of the members' secret keys (Vieta's formulas), so the keys are not really hidden. A small repair is not enough: giving every member a new $\sigma_i$ at every update would need a new login through the TA for all members, and hiding every coefficient would need a secret shared with each member, which is the same as sending each member its own key. This is why C1 drops the polynomial.

### 4.3 F3: no forward secrecy

**Theorem 4.6** (Old keys can be rebuilt)**.** *Suppose an attacker recorded $\xi_i$ and the coefficients of $\Lambda$ in past sessions. If $s_i$ leaks later, the attacker can compute the group key of every past session of UAV $i$.*

*Proof.* $\sigma_i = h_1(s_i \xi_i)P$ depends only on the long-term $s_i$ and the public $\xi_i$. With $s_i$ the attacker computes $\sigma_i$ for each recorded session and then $\kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i$. No value that was deleted after the session is needed. □

**Effect.** *Forward secrecy* (Table II) does not hold. A captured UAV, or a breached TA, opens all the old traffic.

## 5 Our contributions

### 5.1 C1: authentication with the TA offline

**Idea.** The TA works only before the mission. Each UAV has a *certificateless* key [2]: one part comes from the TA and one part is made by the UAV itself. In the field the TUAV checks UAVs alone, and sends each one its own sealed copy of the group key. We use an ordinary elliptic-curve group $\mathbb{G}$ with generator $P$; no pairing is needed.

**Keys (before the mission).** The TA has a master key $s$ and publishes $P_{\text{pub}} = sP$. A UAV picks its own secret $x$ and sends $X = xP$. The TA picks $y$, gives the UAV a pseudonym $PID$ and computes

$$Y = yP, \qquad h = H_1(PID \,\|\, ep \,\|\, X \,\|\, Y), \qquad d = y + s\,h.$$

The UAV keeps $sk = x + d$. Its public key is $pk = X + Y + h\,P_{\text{pub}}$, which anyone can compute from public values. (The pair $(Y, d)$ is a Schnorr signature of the TA, as in [3].)

**Joining (UA1–UA5, Figure 5.1).** The TUAV broadcasts a signed beacon with a fresh value $E_a = e_a P$. UAV $i$ picks fresh $e_i, r_i$ and sends $E_i = e_i P$ with a Schnorr signature [4] $(R_i, v_i)$ on the join message:

$$R_i = r_i P, \qquad c_i = H_2(m_i \,\|\, PID_i \,\|\, X_i \,\|\, Y_i \,\|\, R_i \,\|\, ts_i), \qquad v_i = r_i + c_i\, sk_i.$$

One signature is valid if $v_i P = R_i + c_i\, pk_i$. The TUAV checks all $n$ requests at once with random 64-bit weights $\lambda_i$ [6]:

$$\Big(\sum_i \lambda_i v_i\Big) P = \sum_i \lambda_i R_i + \sum_i \lambda_i c_i\, pk_i. \tag{5.1}$$

If this fails, it checks the requests one by one and keeps the valid ones. For each accepted UAV it computes

$$K_i = \mathsf{KDF}(e_a E_i \,\|\, PID_i \,\|\, ID_a \,\|\, ts_i), \qquad C_i = \mathsf{Enc}(K_i,\ GK \,\|\, \nu),$$

and sends all $C_i$ in one broadcast. UAV $i$ computes the same $K_i$ from $e_i E_a$ and opens its copy of the group key $GK$. Both sides then delete $e_a$ and $e_i$. On every join and every leave the TUAV picks a fresh $GK$ (version $\nu$ goes up) and sends it only to current members.

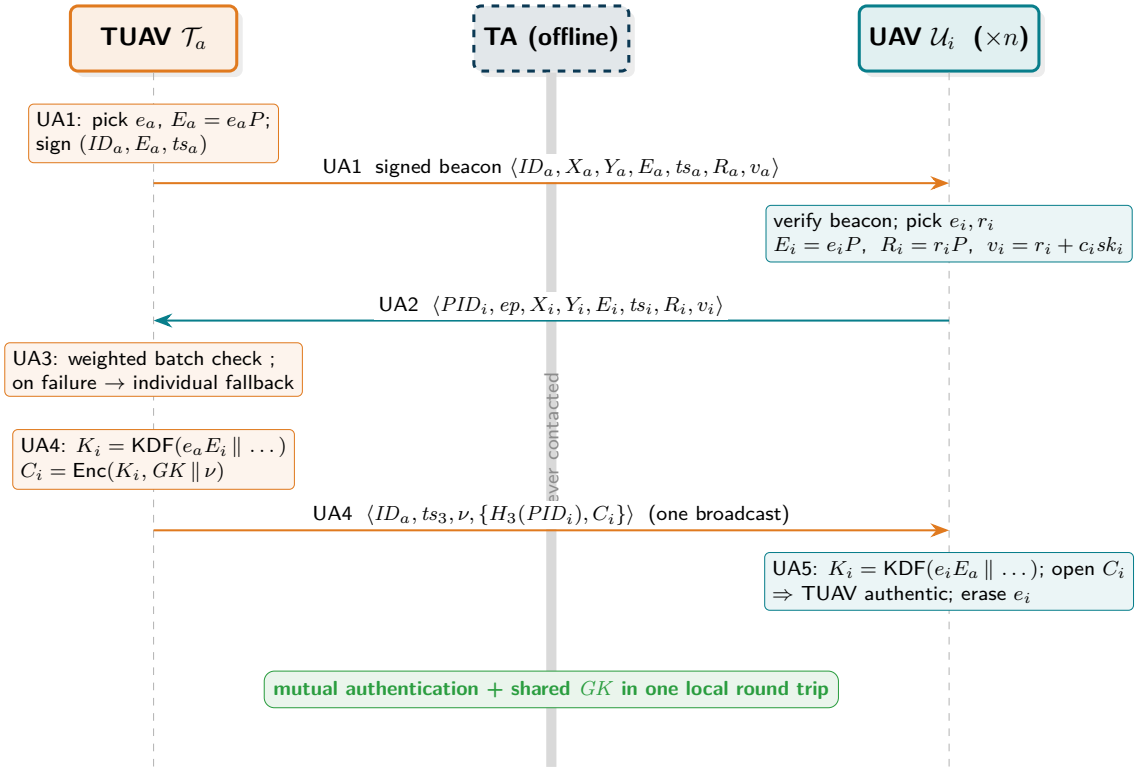



**Figure 5.1:** Message flow of C1 between the TUAV and $n$ UAVs. Time runs from top to bottom; boxes show local computation. The TA (centre) is never contacted. Compared with Figure 3.3, there is no satellite link, no pairing and no CRT polynomial.

**Security.** We state the results and the main reason for each.

**Theorem 5.1** (Security of C1)**.** *Assume the discrete-logarithm (ECDLP) and CDH problems are hard in $\mathbb{G}$, hash functions are random oracles and* Enc *is a secure authenticated encryption scheme. Then:*

*(i)* ***Unforgeability.*** *Nobody can forge a UAV's signature, not even the TA.*

*(ii)* ***Batch soundness.*** *A batch with an invalid request passes Eq.* (5.1) *with probability at most $2^{-64}$.*

*(iii)* ***Group-key secrecy.*** *An outsider, a removed member and a newcomer learn nothing about group keys they should not have.*

*(iv)* ***Forward secrecy.*** *If all long-term keys leak later, old group keys stay secret.*

*Proof sketch.* (i) There are two kinds of attacker. *A malicious TA* knows $s$ and every $d_i$, but not $x_i$. A forgery can be rewound with the forking lemma [5], which gives two signatures with the same $R$ and different $c$; from them one gets $sk_i = x_i + d_i$, and so $x_i$. This solves the ECDLP for $X_i$. *An outsider* does not know $s$. If it forges under keys $(X', Y')$ of its own, the same rewinding gives $d'$ with $d'P = Y' + h'P_{\text{pub}}$. Because $h'$ is a hash that includes $Y'$, a second rewinding gives another

such equation with a different $h'$, and from the two equations one gets $s$. This solves the ECDLP for $P_{\text{pub}}$ (the argument of [3]). (ii) Eq. (5.1) says $\sum_i \lambda_i \Delta_i = 0$, where $\Delta_i = v_i P - R_i - c_i pk_i$ is the error of request $i$. If some $\Delta_j \neq 0$, only one value of $\lambda_j$ makes the sum zero, so the chance is at most $2^{-64}$ [6]. (iii) $GK$ appears only inside the $C_i$. Each $K_i$ needs $e_a e_i P$, which is a CDH problem for everyone except the two ends, and $E_a, E_i$ are signed so they cannot be replaced. Nothing that is published depends on the members' keys, so the attacks of F2 have nothing to work on. (iv) Long-term keys only sign; $K_i$ comes from $e_a, e_i$, which were deleted. □

| **Weakness** | **How C1 removes it** |
|---|---|
| F1 TA impersonation | $sk = x + d$ and the TA never sees $x$. The TA could only issue a second credential for the same pseudonym (Girault level 2 [7]; the base scheme is level 1). |
| F2 CRT leakage | no public polynomial; one sealed copy per member; fresh key on every join and leave |
| F3 no forward secrecy | keys come from fresh $e_a, e_i$ that are deleted after use |

**Cost.** A UAV does about six scalar multiplications to join (two for $E_i$ and $K_i$, one to sign, three to check the beacon). The TUAV needs about $2n + 4$ scalar multiplications for $n$ UAVs and no pairing. With the paper's constants this is about $1.89n + 3.6$ ms, i.e. about 230 ms for 120 UAVs. For large groups the TUAV can use a key tree (LKH [11]), so that a key change needs about $\log n$ messages.

## 5.2 C2: handover between TUAVs

**Gap.** The paper assumes one TUAV "with the effective range covering the entire region" (IS4). A real disaster area needs several TUAVs, and a UAV that moves would have to log in again through the TA.

**Protocol (Figure 5.2).** Neighbouring TUAVs share a link key $K_{ab}$. Before member $\mathcal{M}$ leaves, the old TUAV $\mathcal{T}_a$ makes a handover key $hk = \mathsf{KDF}(K_{\mathcal{M}} \,\|\, \mathtt{HO} \,\|\, ID_b \,\|\, ts)$ and a one-time ticket $Tk = \mathsf{Enc}(K_{ab},\ PID \,\|\, pk \,\|\, hk \,\|\, N \,\|\, exp)$ with a nonce $N$ and an expiry time. $\mathcal{M}$ sends $Tk$, a fresh $E = eP$ and *its own signature* to the new TUAV $\mathcal{T}_b$. $\mathcal{T}_b$ checks the ticket and the signature, derives $K' = \mathsf{KDF}(e_b E \,\|\, hk \,\|\, PID \,\|\, ID_b)$, changes its cell key, and answers with $E_b$, the new cell key and *its own signature*. The old TUAV changes its key after $\mathcal{M}$ has left.

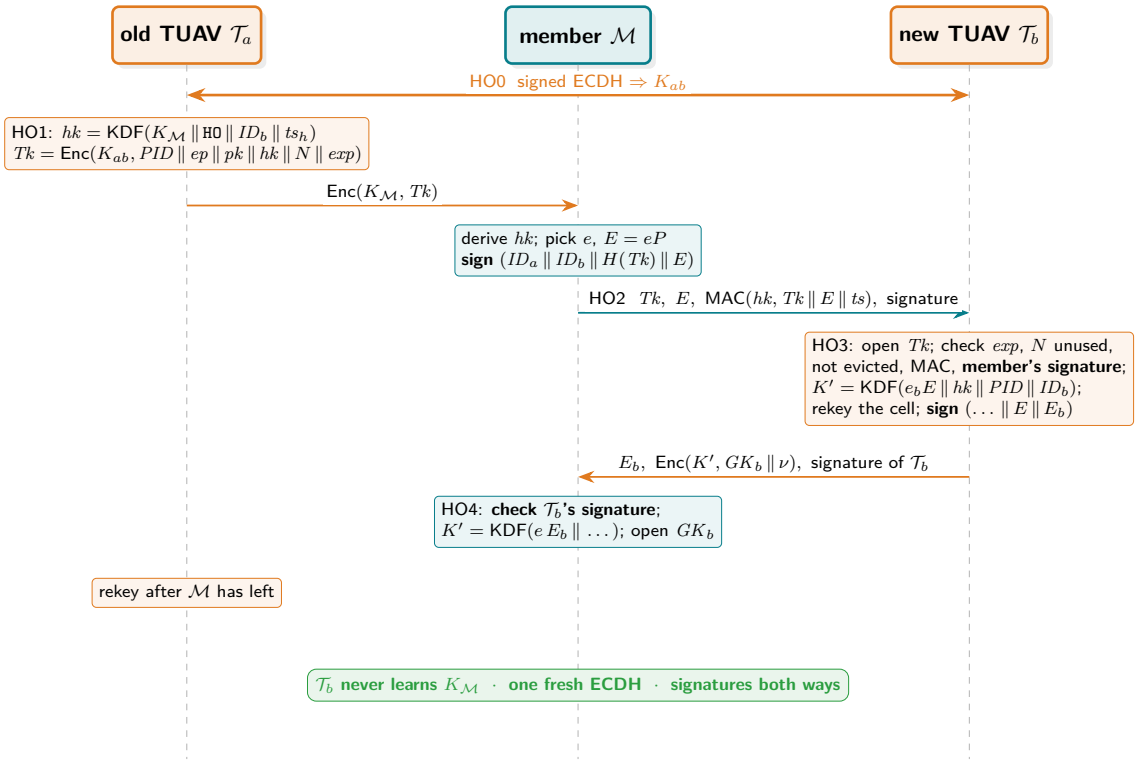



**Figure 5.2:** Message flow of the C2 handover between the old TUAV $\mathcal{T}_a$, the moving member $\mathcal{M}$ and the new TUAV $\mathcal{T}_b$. Time runs from top to bottom. The two signatures in bold (by the member in HO2 and by the new TUAV in HO3) are needed so that a captured old TUAV cannot misuse a ticket.

**Why each part is needed.**

- *$hk$ instead of $K_{\mathcal{M}}$*: the new TUAV never gets the member's old key.
- *Fresh values $E$, $E_b$*: the new key $K'$ stays secret even if $K_{ab}$ or $hk$ leaks later.
- *Nonce and expiry*: a ticket works only once and only for a short time.
- *Member's signature*: a captured old TUAV cannot create tickets for members who did not ask to move.
- *New TUAV's signature*: a captured old TUAV, which knows $hk$, cannot answer in the name of the new TUAV.

**Security.** We modelled the protocol in ProVerif [17] with a network attacker who also controls a *captured old TUAV*. With both signatures, five properties hold: each side is sure of the other (two properties), $K'$ is secret, the new cell key is secret, and $K'$ stays secret if $K_{ab}$ and $hk$ leak later. Without the member's signature, or without the new TUAV's signature, the tool finds an attack. The new TUAV never learns the member's old key $K_{\mathcal{M}}$. No message goes to the TA.

## 5.3 C3: voting out a compromised TUAV

**Gap.** After C1 the TUAV is the trusted party of its cell. If its software is hacked, nobody in the field can remove it, because the TA is offline.

**Protocol (Figure 5.3).** A neighbouring TUAV that has evidence of misbehaviour (for example two conflicting signed messages) signs a vote "evict $T$". A set of votes is a valid **eviction certificate** if: (V1) it has at least $t$ different TUAV signers; (V2) the target is not a signer; (V3) all votes name the same target; (V4) all signatures pass the batch check (5.1). Every UAV can check this with public keys. A UAV that receives a valid certificate against its TUAV leaves it for good and joins a neighbour with the normal join (C1).

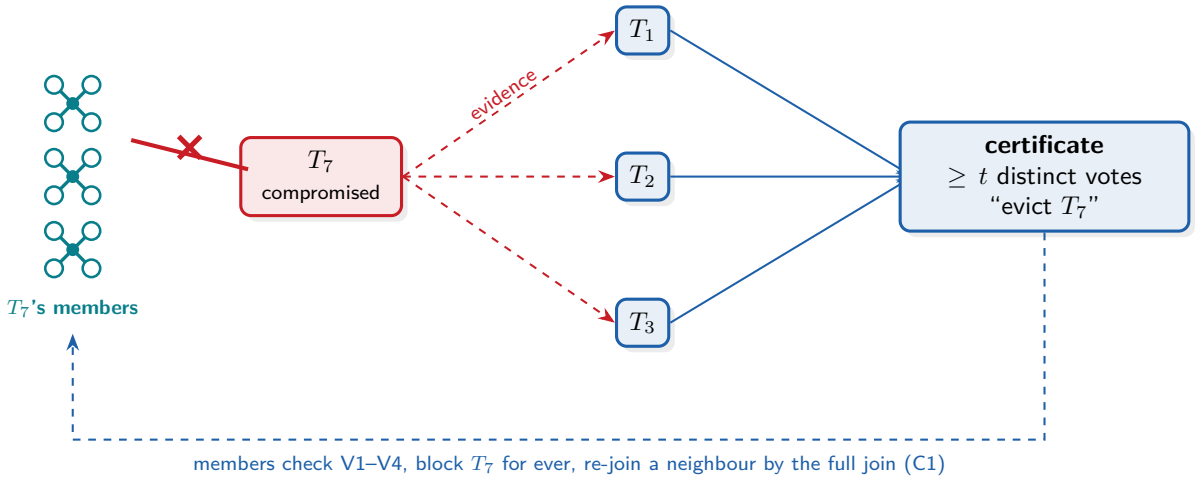



**Figure 5.3:** Quorum eviction (C3). The neighbours $T_1, T_2, T_3$ get evidence against the compromised TUAV $T_7$ (dashed arrows) and each signs a vote. At least $t$ different votes form an eviction certificate. It is delivered to the members of $T_7$, who check it and join a neighbour.

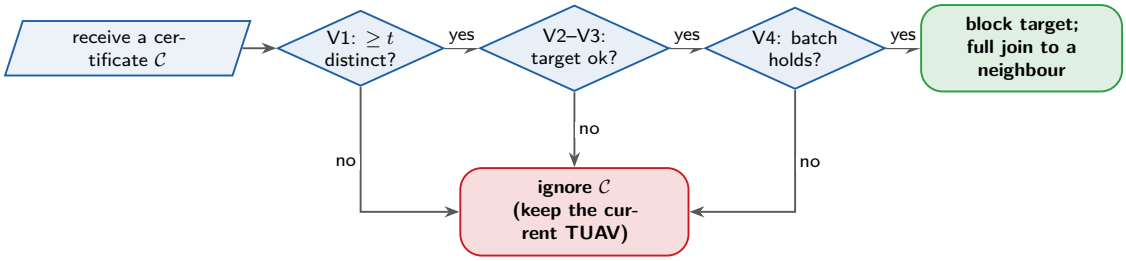



**Figure 5.4:** Flowchart of how a UAV checks an eviction certificate against the four conditions V1–V4. Only if all conditions hold does the UAV leave its TUAV; otherwise it ignores the certificate.

**Theorem 5.2** (Eviction safety and liveness)**.** *(Safety) If at most $t-1$ TUAVs are compromised and honest TUAVs vote only on evidence, no honest TUAV can be evicted. (Liveness) If $t$ honest TUAVs have evidence, a valid certificate is formed.*

*Proof.* A certificate needs $t$ valid votes from different TUAVs. At most $t-1$ come from compromised ones, and a vote of an honest TUAV cannot be forged (Theorem 5.1). So at least one honest TUAV voted, and it votes only on evidence. Liveness: $t$ honest votes satisfy V1–V4. □

This is the same idea as the quorum certificate in PBFT [12]. We use it in a new place: removing a key-management node while the authority is offline. With up to $f$ bad neighbours, choose $t = f+1$.

## 5.4 C4: the tether-split key

**Gap.** The TUAV is the most valuable drone to capture. In the base scheme and in C1, a captured TUAV holds its whole key and can keep acting as a TUAV. C3 does not help quickly here, because voting needs evidence and time.

**Idea (Figure 5.5).** The TUAV's signing key is split into two shares, $sk = a + g$. The drone holds $a$. A small secure chip in the ECRV holds $g$. The public key $pk = aP + gP$ is the same as before. The two sides sign together over the tether:

$$R = R_A + R_G, \qquad c = H_2(m \,\|\, \ldots \,\|\, R), \qquad v = (r_A + c\,a) + (r_G + c\,g),$$

where $R_A = r_A P$ is the drone's part and $R_G = r_G P$ the chip's part. The drone must first send a commitment $H(R_A)$, and the chip runs one session at a time. The result satisfies $vP = R + c\,pk$, so

it is a normal signature and the UAVs need no change. If the tether is cut, the chip stops signing and the ECRV broadcasts a "this TUAV is out of service" message that both shares signed in advance.

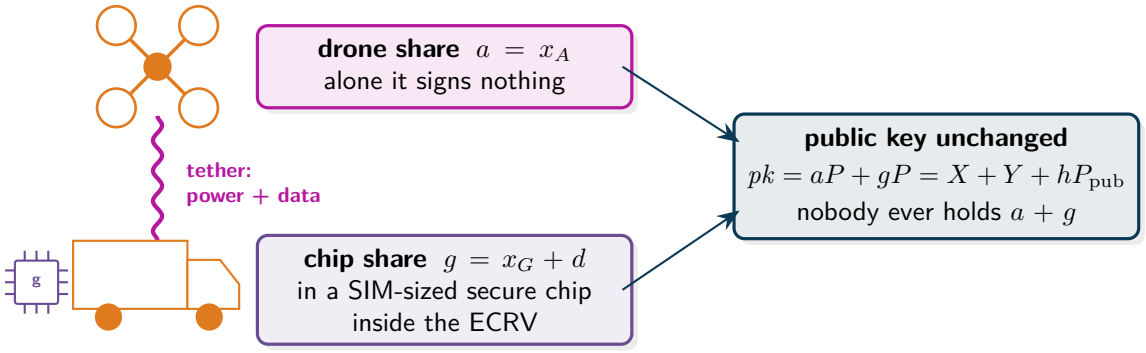



**Figure 5.5:** The tether-split key (C4). The drone holds the share $a$ and a secure chip in the ECRV holds the share $g$; they are connected by the tether. The public key $pk = aP + gP$ is unchanged, and no single device holds the full key $a + g$.

**Signing steps (Figure 5.6).** (J1) The drone picks $r_A$ and sends the message $m$ and the commitment $H(R_A)$ to the chip. (J2) The chip checks that the tether is alive, that $m$ is an allowed type of message and that no other session is open; it picks $r_G$ and sends $R_G$. (J3) The drone sends $R_A$, and the chip checks it against the commitment. (J4) The chip sends $v_G = r_G + c\,g$. (J5) The drone computes $v = r_A + c\,a + v_G$.

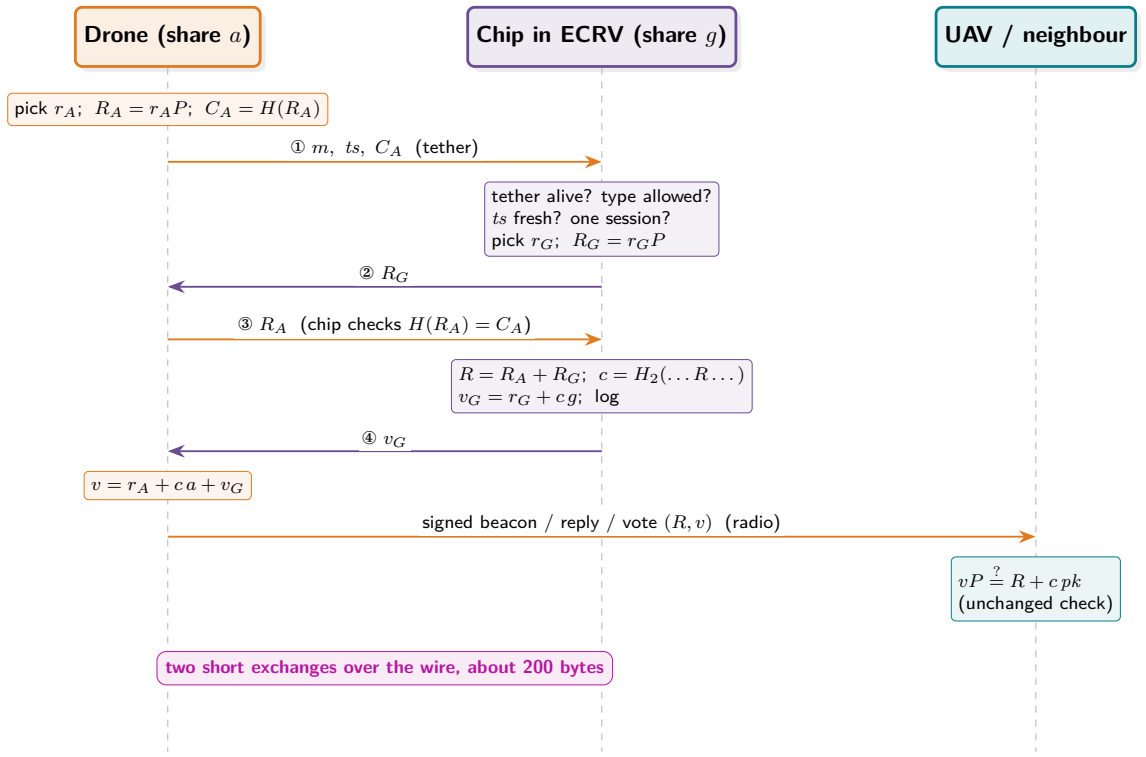



**Figure 5.6:** Joint signing over the tether (steps J1–J5). Messages 1–4 travel on the tether between the drone and the chip. The final signature $(R, v)$ is sent by radio and is checked in the usual way. The commitment in message 1 forces the drone to fix its value $R_A$ before it sees the chip's.

**Theorem 5.3** (Half a key cannot sign)**.** *Let $G = gP = pk - aP$. If an attacker who knows the drone's share $a$ produces a valid signature $(R, v)$ on a message the chip did not sign, then $(R,\ v - c\,a)$ is a valid Schnorr signature for the public key $G$, whose secret key the attacker does not know.*

*Proof.* $(v - c\,a)P = vP - c\,aP = R + c\,(aP + G) - c\,aP = R + c\,G.$ □

So forging with a captured drone is as hard as forging a Schnorr signature without the key, which needs the ECDLP. The commitment step is needed because signing many sessions in parallel without it is unsafe (the ROS attack [16]); the standard FROST protocol [15] is an alternative.

**Theorem 5.4** (A captured drone cannot sign new messages)**.** *Assume the chip uses the commitment step and one session at a time. Then an attacker who controlled the drone in earlier sessions and later gets the share a still cannot sign a message that the chip did not approve, unless it solves the ECDLP (random-oracle model).*

*Proof sketch.* Put an ECDLP challenge in $G = gP$. The chip can be simulated without $g$: because the drone commits to $R_A$ first, the simulator knows $R_A$, picks $v_G$ and $c$, sets $R_G = v_G P - c\,G$ and programs the hash so that $c$ is the challenge for $R = R_A + R_G$ (the method used for Schnorr multi-signatures). A forgery then gives a signature under $G$ by Theorem 5.3, and the forking lemma extracts $g$. □

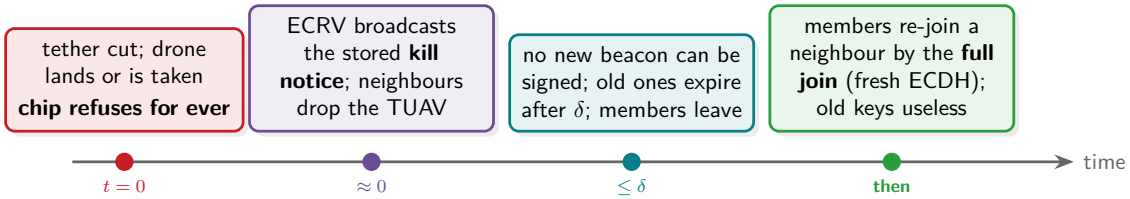



**Figure 5.7:** What happens after the tether is cut. The chip stops signing at once, the ECRV broadcasts the stored "out of service" message, old beacons expire after a short period $\delta$, and the UAVs join a neighbouring TUAV with fresh keys.

**Cost.** With the paper's constants, one TUAV signature costs 0.903 ms today. With the split the drone needs 0.915 ms and the chip 0.904 ms at the same time. The tether carries about 200 bytes per signature. The ECRV needs only a small chip that stores 32 bytes and does one scalar multiplication per signature.

**What is new.** Splitting a key between a device and a helper that can switch it off is known as mediated cryptography [13, 14]. New here is the place: the helper is the ECRV at the other end of the tether, and the switch is the tether itself. **Limit:** if the ECRV is seized together with the drone, both shares are lost.

## 5.5 A1–A7: fixing our own first design

Before finalising the design we attacked our own first version and found seven flaws.

| | Flaw in our first design | Fix |
|---|---|---|
| A1 | one forged packet naming a UAV got that UAV evicted | a failed signature never evicts anybody |
| A2 | a captured TUAV could create handover tickets | the member signs its own handover request |
| A3 | a captured TUAV could frame a member at a neighbour | evictions cross cells only with evidence that can be checked |
| A4 | a credential for one role could be used for another | the role is bound into the identity ($ep$ inside $h$) |
| A5 | an old key update could be replayed | key versions may only increase |
| A6 | a newcomer received the key already in use | new key on every arrival |
| A7 | a captured old TUAV could act as the new TUAV | the new TUAV signs its reply (found by ProVerif) |

# 6 Results

## 6.1 Security comparison

| Property | Base paper | Ours | Shown in |
|---|---|---|---|
| The TA cannot sign as a UAV | ✗ | ✓ | Thm. 4.1, 5.1 |
| A removed UAV cannot read new keys | ✗ | ✓ | Cor. 4.3, Thm. 5.1 |
| A new UAV cannot read old keys | ✗ | ✓ | Cor. 4.3, Thm. 5.1 |
| An outsider learns nothing from the broadcasts | ✗ | ✓ | Thm. 4.4, 5.1 |
| Forward secrecy | ✗ | ✓ | Thm. 4.6, 5.1 |
| Login works with the TA offline | ✗ | ✓ | Section 5.1 |
| Handover between TUAVs | – | ✓ | Section 5.2 |
| Removing a compromised TUAV | – | ✓ | Thm. 5.2 |
| A captured TUAV cannot act as the TUAV | ✗ | ✓ | Thm. 5.3 |

## 6.2 Cost from the paper's own constants

The paper gives the time of each operation (MIRACL library): pairing 7.2232 ms, scalar multiplication in $\mathbb{G}_1$ 2.6784 ms, scalar multiplication on the ordinary curve 0.9018 ms. With these numbers:

| For $n$ UAVs | Base paper | Ours (C1) |
|---|---|---|
| Pairings | $3n$ (at the TA) | 0 |
| Satellite round trips per login | 1 | 0 |
| Time for $n = 120$ (TA + TUAV) | $\approx 3720$ ms + satellite | $\approx 230$ ms |
| Key-distribution traffic | $\approx 10n^3$ bytes | $\approx 80n$ bytes |
| Extra drone work per signature with C4 | – | +0.012 ms |

The traffic of the base scheme is large because coefficient $a_k$ has about $160(n-k)$ bits, so one coefficient set is about $10n^2$ bytes, and it is sent to each of the $n$ UAVs.

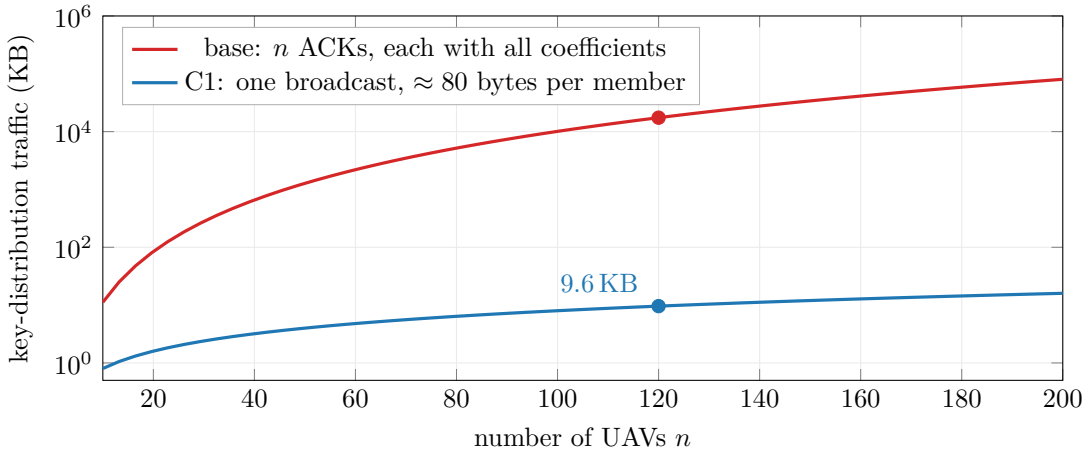



**Figure 6.1:** Traffic needed to distribute one group key against the number of UAVs $n$ (logarithmic vertical axis). Red: the base scheme, which sends all polynomial coefficients to each UAV. Blue: C1, one broadcast with about 80 bytes per member. The marked points are the values for $n = 120$.

## 6.3 Simulator and implementation

We built two programs, both in Python with real BN128 pairings. The code is public:

https://github.com/swarajkumar001/
Secure-and-Efficient-Authenticated-Key-Management-Scheme-for-UAV-Assisted-Infrastructure-Less-IoVs

- **Simulator of the base scheme** (folder `uav_iov_simulator`). A web application that runs the paper's steps IS, US, GA, GD and DU with the same equations, and shows the network, the messages and the group key (Figure 6.2).
- **Implementation of our design and experiments** (folder `final`). It contains both schemes, the attacks, and the ProVerif models.

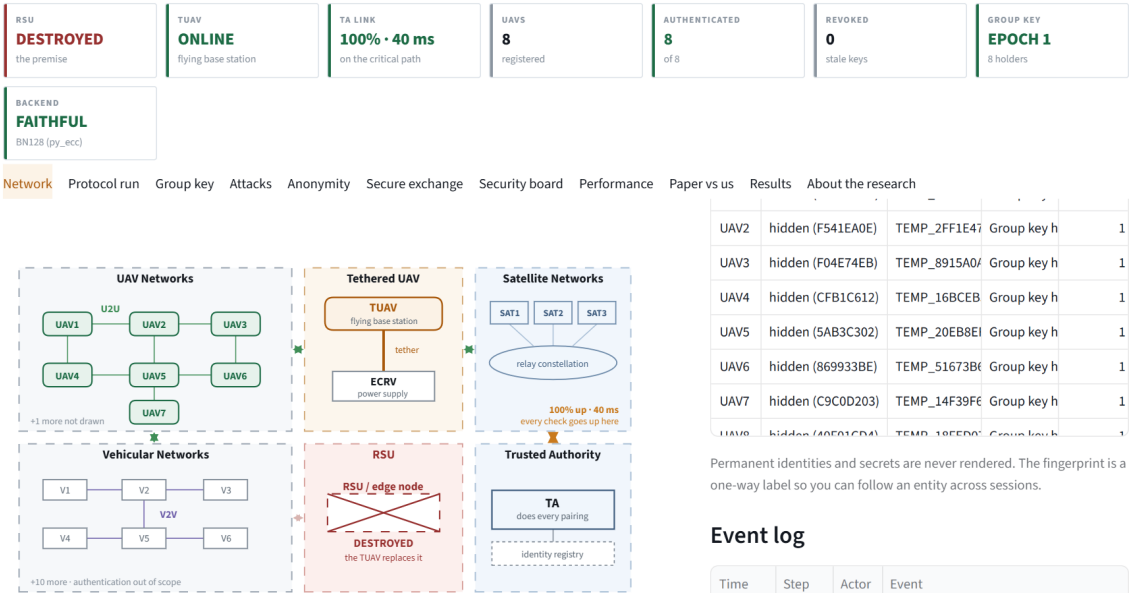



**Figure 6.2:** Screenshot of our simulator of the base scheme. Top: status of the system (RSU destroyed, TUAV online, 8 UAVs registered and authenticated, group key of epoch 1). Left: the network as in the paper's system model. Right: the registered UAVs with hidden permanent identities and temporary identities.

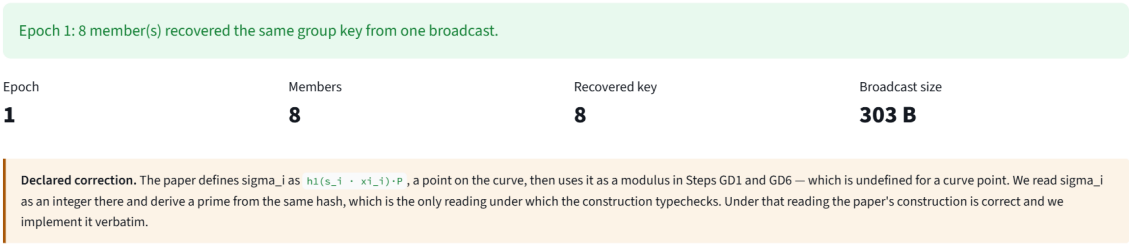



**Figure 6.3:** Screenshot of the simulator after group key distribution (steps GD1–GD6). All 8 members recover the same group key from one broadcast of 303 bytes. The note at the bottom records that we read $\sigma_i$ as an integer, because the paper uses it as a modulus.

| **Measured on our implementation (same laptop, same code base)** | **Base paper** | **Ours** |
|---|---|---|
| TA forges a UAV's request | succeeds | fails |
| Removed UAV computes the new group key | succeeds | fails |
| Time to authenticate and key 100 UAVs | 387.6 s | 6.5 s ($\approx 60\times$ faster) |
| Key-distribution traffic for 120 UAVs | 17.5 MB | 9.6 KB |
| Flaws A1–A7 that still work (first design $\rightarrow$ final design) | – | 7 of 7 $\rightarrow$ 0 of 7 |
| Handover properties verified in ProVerif | – | 5 of 5 |

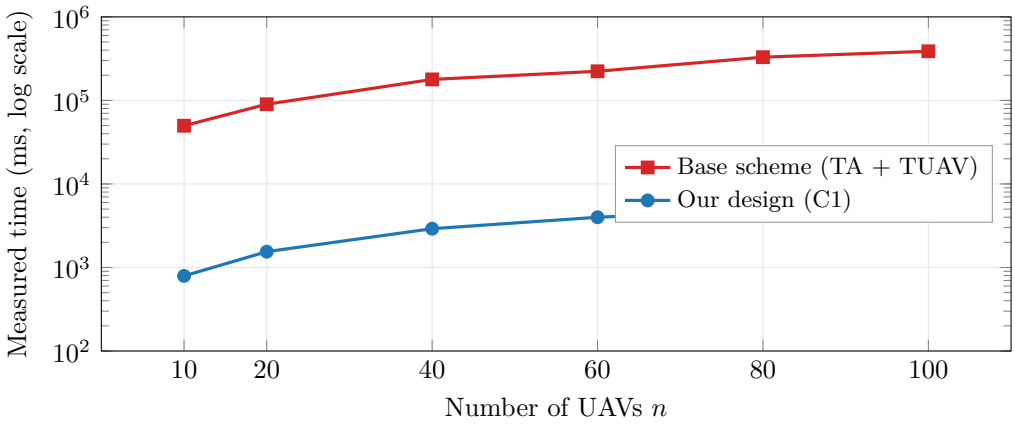



**Figure 6.4:** Measured time to authenticate $n$ UAVs and give them the group key, on our implementation of both schemes (same laptop, pure Python, BN128 curve). The vertical axis is logarithmic. The base scheme needs three pairings per UAV at the TA; our design needs none, and is about 60 times faster at every size.

The times are from pure Python, so only the ratio is meaningful, not the absolute seconds.

# 7 Related work and conclusion

## 7.1 Related work

**Group keys with CRT or polynomials.** Sending one broadcast that every member can open with its own key goes back to the "secure lock" of Chiou and Chen [8]. Sharma and Purushothama [9] show that known CRT-based group-key schemes lose backward secrecy when a user is corrupted. Our insider result is one more case of this weakness. Our outsider result is different: it needs no corrupted user. The standard way to change keys in large groups with a separate key per member is the key tree (LKH) [11].

**Certificateless signatures.** Al-Riyami and Paterson [2] introduced keys that have one part from the authority and one part from the user, so there is no key escrow. Our credential follows the Schnorr-based scheme of Galindo and Garcia [3], and the batch check follows Bellare, Garay and Rabin [6].

**Quorum decisions.** C3 uses the idea of a quorum certificate from PBFT [12]. Because an eviction is never undone, $f + 1$ votes are enough to stop $f$ bad TUAVs from evicting an honest one.

**Split keys.** In mediated cryptography [13, 14] a key is split between a user and a helper who can stop it at once. Two-party and threshold Schnorr signing is standardised as FROST [15], and the ROS attack [16] shows why parallel sessions must be handled with care. We did not find earlier work that splits a tethered drone's key across its tether.

| Approach | How the group key reaches members | Authority online at login? | Removed member locked out? |
|---|---|---|---|
| CRT broadcast (secure lock [8], base paper [1]) | one public broadcast built from all members' keys | yes (base paper) | no ([9], our F2) |
| Key tree (LKH [11]) | separate keys, about $\log n$ messages per change | needs a key server | yes |
| Our design (C1–C4) | one sealed copy per member, fresh key on every change | no | yes |

### 7.2 Limitations

- The outsider attack needs two broadcasts; a single broadcast and mixed updates are open.
- The TA in C1 can still register a second credential for a pseudonym (level 2, not level 3).
- The security of C1 and C4 is given as proof sketches; C3 and C4 do not yet have a formal model, and C4 is not yet built on real hardware.
- If $t$ or more TUAVs are compromised, or the ECRV is seized with the drone, C3 and C4 do not help.

### 7.3 Conclusion

The base paper has a good idea: a tethered drone instead of a broken roadside unit. But its security depends on an authority that knows every secret and on a public polynomial that leaks the keys. We proved three weaknesses from the paper's own equations. Our design takes the authority out of the field, gives each member its own sealed key, and protects the TUAV that now holds the trust: UAVs move with signed tickets, TUAVs can vote out a bad TUAV, and a captured TUAV cannot sign because half of its key stays in the vehicle. Future work is a formal model of C4, a test on real drones, and the open cases of the outsider attack.

## References


[1] H. Tan, W. Zheng and P. Vijayakumar, "Secure and efficient authenticated key management scheme for UAV-assisted infrastructure-less IoVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 6389–6400, 2023.

[2] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *ASIACRYPT 2003*, pp. 452–473.

[3] D. Galindo and F. D. Garcia, "A Schnorr-like lightweight identity-based signature scheme," in *AFRICACRYPT 2009*, pp. 135–148.

[4] C. P. Schnorr, "Efficient signature generation by smart cards," *J. Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.

[5] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptology*, vol. 13, no. 3, pp. 361–396, 2000.

[6] M. Bellare, J. A. Garay and T. Rabin, "Fast batch verification for modular exponentiation and digital signatures," in *EUROCRYPT 1998*, pp. 236–250.

[7] M. Girault, "Self-certified public keys," in *EUROCRYPT 1991*, pp. 490–497.

[8] G.-H. Chiou and W.-T. Chen, "Secure broadcasting using the secure lock," *IEEE Trans. Softw. Eng.*, vol. 15, no. 8, pp. 929–934, 1989.

[9] P. Sharma and B. R. Purushothama, "Proving the (in)security of CRT based key management schemes under SAOA model," *Wireless Pers. Commun.*, vol. 134, no. 3, pp. 1299–1321, 2024.

[10] A. K. Lenstra, H. W. Lenstra Jr. and L. Lovász, "Factoring polynomials with rational coefficients," *Math. Ann.*, vol. 261, pp. 515–534, 1982.

[11] C. K. Wong, M. Gouda and S. S. Lam, "Secure group communications using key graphs," in *ACM SIGCOMM 1998*, pp. 68–79.

[12] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *OSDI 1999*, pp. 173–186.

[13] D. Boneh, X. Ding, G. Tsudik and C. M. Wong, "A method for fast revocation of public key certificates and security capabilities," in *USENIX Security Symposium*, 2001.

[14] S. S. M. Chow, C. Boyd and J. M. González Nieto, "Security-mediated certificateless cryptography," in *PKC 2006*, pp. 508–524.

[15] D. Connolly, C. Komlo, I. Goldberg and C. A. Wood, "The Flexible Round-Optimized Schnorr Threshold (FROST) protocol for two-round Schnorr signatures," RFC 9591, 2024.

[16] F. Benhamouda, T. Lepoint, J. Loss, M. Orrù and M. Raykova, "On the (in)security of ROS," in *EUROCRYPT 2021*.

[17] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Found. Trends Priv. Secur.*, vol. 1, no. 1–2, pp. 1–135, 2016.